Task 1

```
import tensorflow as tf
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import random
import os

SEED = 42


random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)
os.environ["PYTHONHASHSEED"] = str(SEED)

print("TensorFlow Version :", tf.__version__)
print("NumPy Version      :", np.__version__)
print("Pandas Version     :", pd.__version__)
print("Matplotlib Version :", plt.matplotlib.__version__)

gpu = tf.config.list_physical_devices('GPU')

if gpu:
    print("\nGPU Available")
    print(gpu)
else:
    print("\nGPU NOT Available")
    print("CPU is slower because it processes operations
sequentially.")
    print("GPU performs parallel matrix operations, ideal for CNN
training.")
```

```
TensorFlow Version : 2.19.0
NumPy Version      : 2.0.2
Pandas Version     : 2.2.2
Matplotlib Version : 3.10.0

GPU NOT Available
CPU is slower because it processes operations sequentially.
GPU performs parallel matrix operations, ideal for CNN training.
```

```
from tensorflow.keras.datasets import mnist, cifar10
```

```
(x_train_mnist, y_train_mnist), (x_test_mnist, y_test_mnist) =
mnist.load_data()
(x_train_cifar, y_train_cifar), (x_test_cifar, y_test_cifar) =
cifar10.load_data()


print("MNIST Train Shape :", x_train_mnist.shape)
print("MNIST Test Shape  :", x_test_mnist.shape)

print("CIFAR Train Shape :", x_train_cifar.shape)
print("CIFAR Test Shape  :", x_test_cifar.shape)


print("\nMNIST dtype :", x_train_mnist.dtype)
print("MNIST range :", x_train_mnist.min(), "to", x_train_mnist.max())

print("\nCIFAR dtype :", x_train_cifar.dtype)
print("CIFAR range :", x_train_cifar.min(), "to", x_train_cifar.max())


unique, counts = np.unique(y_train_mnist, return_counts=True)

print("\nMNIST Samples per class:")
for u, c in zip(unique, counts):
    print(f"Class {u}: {c}")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-
python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 4s 0us/step
MNIST Train Shape : (60000, 28, 28)
MNIST Test Shape  : (10000, 28, 28)
CIFAR Train Shape : (50000, 32, 32, 3)
CIFAR Test Shape  : (10000, 32, 32, 3)

MNIST dtype : uint8
MNIST range : 0 to 255

CIFAR dtype : uint8
CIFAR range : 0 to 255

MNIST Samples per class:
Class 0: 5923
Class 1: 6742
Class 2: 5958
Class 3: 6131
Class 4: 5842
Class 5: 5421
Class 6: 5918
```

```
Class 7: 6265
Class 8: 5851
Class 9: 5949
```

2 B

```
class_names = ['airplane','automobile','bird','cat','deer',
               'dog','frog','horse','ship','truck']

fig, axes = plt.subplots(2, 10, figsize=(18, 5))

mnist_idx = np.random.choice(len(x_train_mnist), 10)
cifar_idx = np.random.choice(len(x_train_cifar), 10)

for i, idx in enumerate(mnist_idx):
    axes[0, i].imshow(x_train_mnist[idx], cmap='gray')
    axes[0, i].set_title(f"{y_train_mnist[idx]}")
    axes[0, i].axis('off')

for i, idx in enumerate(cifar_idx):
    axes[1, i].imshow(x_train_cifar[idx])
    axes[1, i].set_title(class_names[y_train_cifar[idx][0]])
    axes[1, i].axis('off')

plt.tight_layout()
plt.savefig("dataset_samples.png")
plt.show()
```

```
def preprocess_mnist(images, labels):
    print("Before Processing")
    print("Shape :", images.shape)
    print("Range :", images.min(), images.max())


    images = images.astype('float32') / 255.0
```

```
    images = np.expand_dims(images, axis=-1)


    labels = tf.keras.utils.to_categorical(labels, 10)

    print("\nAfter Processing")
    print("Shape :", images.shape)
    print("dtype :", images.dtype)
    print("Range :", images.min(), images.max())
    print("Label Shape :", labels.shape)

    return images, labels

x_train_mnist_processed, y_train_mnist_processed = preprocess_mnist(
    x_train_mnist, y_train_mnist
)
```

```
Before Processing
Shape : (60000, 28, 28)
Range : 0 255

After Processing
Shape : (60000, 28, 28, 1)
dtype : float32
Range : 0.0 1.0
Label Shape : (60000, 10)
```

3 B

```
def preprocess_cifar(images, labels):
    print("Before Processing")
    print("Shape :", images.shape)

    images = images.astype('float32') / 255.0
    labels = tf.keras.utils.to_categorical(labels, 10)

    print("\nAfter Processing")
    print("Shape :", images.shape)
    print("dtype :", images.dtype)
    print("Range :", images.min(), images.max())

    return images, labels

x_train_cifar_processed, y_train_cifar_processed = preprocess_cifar(
    x_train_cifar, y_train_cifar
)
```

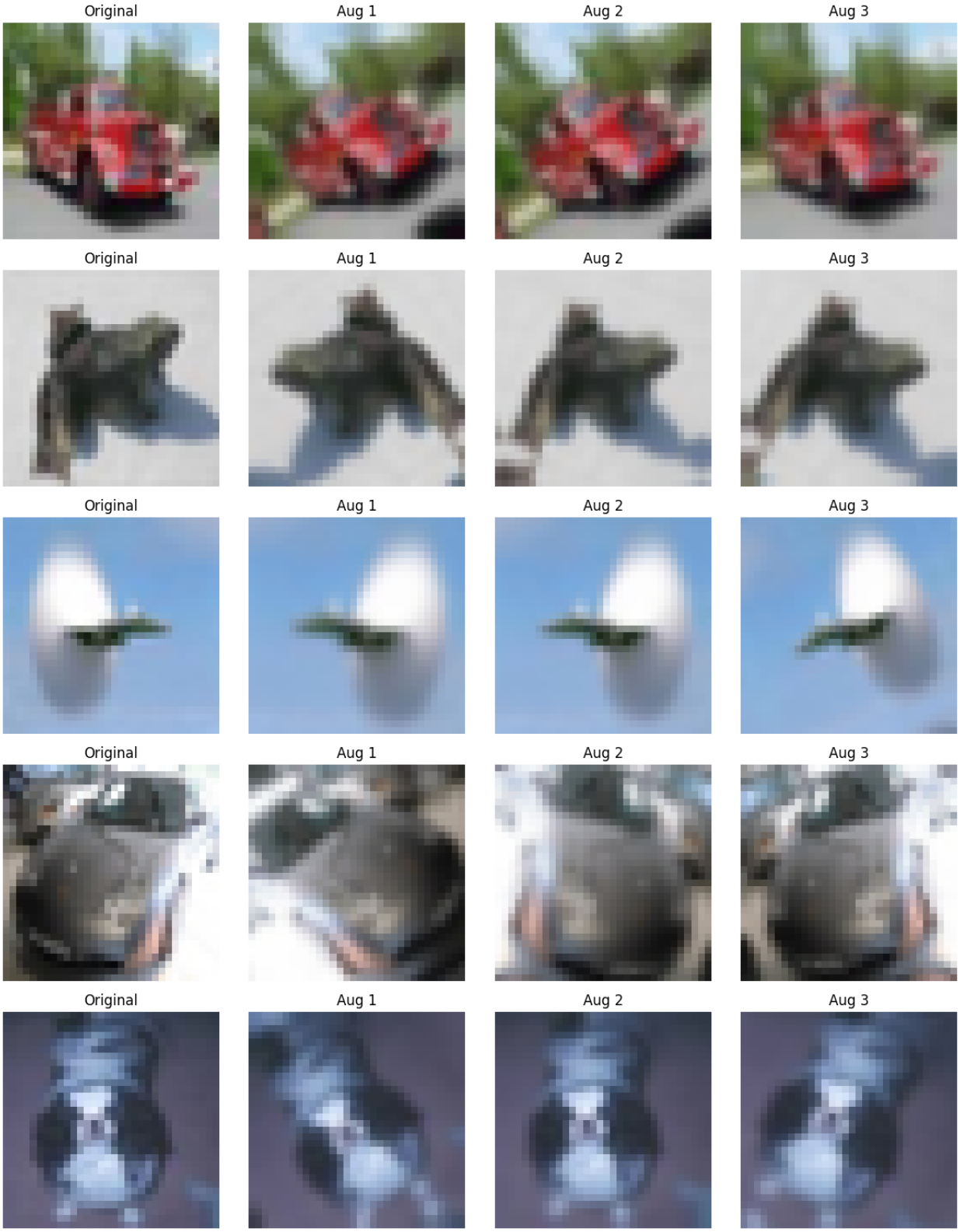
```
Before Processing
Shape : (50000, 32, 32, 3)
```

```
After Processing
Shape : (50000, 32, 32, 3)
dtype : float32
Range : 0.0 1.0
```

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.1),
    tf.keras.layers.RandomZoom(0.1)
])
```

4B

```
fig, axes = plt.subplots(5, 4, figsize=(12, 15))

sample_indices = np.random.choice(len(x_train_cifar_processed), 5)

for row, idx in enumerate(sample_indices):
    original = x_train_cifar_processed[idx]


    axes[row, 0].imshow(original)
    axes[row, 0].set_title("Original")
    axes[row, 0].axis('off')


    for col in range(1, 4):
        augmented = data_augmentation(tf.expand_dims(original, 0))
        axes[row, col].imshow(augmented[0])
        axes[row, col].set_title(f"Aug {col}")
        axes[row, col].axis('off')

plt.tight_layout()
plt.savefig("augmentation_demo.png")
plt.show()
```



There is a section called ANALYSIS & REFLECTION QUESTIONS

Q1

Channel dimension means number of channels.

Grayscale → 1 RGB → 3

Q2

Use mini-batches, patch extraction, and data generators.

Q3

This causes data leakage. Normalization should be based only on training data.

TASK 2

```
import numpy as np

def conv2d(image, kernel, stride=1, padding=0):

    if padding > 0:
        image = np.pad(
            image,
            ((padding, padding), (padding, padding)),
            mode='constant'
        )

    img_h, img_w = image.shape
    ker_h, ker_w = kernel.shape

    out_h = ((img_h - ker_h) // stride) + 1
    out_w = ((img_w - ker_w) // stride) + 1

    output = np.zeros((out_h, out_w))


    for i in range(out_h):
        for j in range(out_w):
            patch = image[
                i*stride:i*stride+ker_h,
                j*stride:j*stride+ker_w
            ]
            output[i, j] = np.sum(patch * kernel)

    return output



image = np.array([
    [3,1,0,2,4],
    [1,5,3,2,1],
    [0,2,6,4,3],
```

```
    [2,3,1,5,2],
    [1,0,2,3,4]
])


kernel = np.array([
    [-1,0,1],
    [-2,0,2],
    [-1,0,1]
])

output = conv2d(image, kernel, stride=1, padding=0)

print("Output Feature Map:")
print(output)
print("Output Shape:", output.shape)
```

```
Output Feature Map:
[[ 7. -3. -3.]
 [13.  3. -7.]
 [ 5.  9.  1.]]
Output Shape: (3, 3)
```

(a) Input = 28×28, Kernel = 5×5, Padding = 0, Stride = 1

Output = floor((Input - Kernel + 2×Padding)/Stride) + 1 = floor((28 - 5 + 2×0)/1) + 1 = floor(23/1) + 1 = 23 + 1 = 24

Final Output Size = 24×24

(b) Input = 28×28, Kernel = 3×3, Padding = 1, Stride = 1

Output = floor((28 - 3 + 2×1)/1) + 1 = floor(27/1) + 1 = 27 + 1 = 28

Final Output Size = 28×28

(c) Input = 32×32, Kernel = 3×3, Padding = 0, Stride = 2

Output = floor((32 - 3 + 2×0)/2) + 1 = floor(29/2) + 1 = 14 + 1 = 15

Final Output Size = 15×15

(d) Two Conv Layers

Layer 1: Input = 32×32 K = 3, P = 1, S = 1

Output = floor((32 - 3 + 2×1)/1) + 1 = 32

Layer 2: Input = 32×32 K = 3, P = 0, S = 1

Output = floor((32 - 3 + 0)/1) + 1 = 30

Final Output Size = 30×30

```python
import tensorflow as tf
from tensorflow.keras import layers, models

lenet = models.Sequential([
    layers.Conv2D(6, (5,5), activation='tanh',
                  input_shape=(28,28,1), padding='valid'),
    layers.AveragePooling2D((2,2), strides=2),

    layers.Conv2D(16, (5,5), activation='tanh'),
    layers.AveragePooling2D((2,2), strides=2),

    layers.Flatten(),

    layers.Dense(120, activation='tanh'),
    layers.Dense(84, activation='tanh'),
    layers.Dense(10, activation='softmax')
])

lenet.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

```
Model: "sequential_1"

┏━━━
```

```
──────┤
│ average_pooling2d_1              │ (None, 4, 4, 16)        │
0 │
│ (AveragePooling2D)               │                         │
│
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ flatten (Flatten)                │ (None, 256)             │
0 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dense (Dense)                    │ (None, 120)             │
30,840 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dense_1 (Dense)                  │ (None, 84)              │
10,164 │
├──────────────────────────────────┼────────────────────────┼──────────
──────┤
│ dense_2 (Dense)                  │ (None, 10)              │
850 │
└──────────────────────────────────┴────────────────────────┴──────────
──────┘

 Total params: 44,426 (173.54 KB)

 Trainable params: 44,426 (173.54 KB)

 Non-trainable params: 0 (0.00 B)
```

First Conv Layer Parameters:

Formula: (K × K × C_in + 1) × C_out

= (5 × 5 × 1 + 1) × 6 = (25 + 1) × 6 = 26 × 6 = 156

```
custom_cnn = models.Sequential([


    layers.Conv2D(32, (3,3), padding='same',
                  input_shape=(32,32,3)),
    layers.BatchNormalization(),
    layers.ReLU(),
    layers.MaxPooling2D((2,2)),


    layers.Conv2D(64, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.ReLU(),
```

```
    layers.MaxPooling2D((2,2)),


    layers.Conv2D(128, (3,3), padding='same'),
    layers.BatchNormalization(),
    layers.ReLU(),
    layers.MaxPooling2D((2,2)),


    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.5),
    layers.Dense(256, activation='relu'),
    layers.Dense(10, activation='softmax')
])

custom_cnn.summary()
```

```
Model: "sequential_2"
```

```
┏━━━
```

```
│                                      │                           │
├──────────────────────────────────────┼───────────────────────────┼──────────
──────┤
│ re_lu_1 (ReLU)                       │ (None, 16, 16, 64)        │
0 │
├──────────────────────────────────────┼───────────────────────────┼──────────
──────┤
│ max_pooling2d_1 (MaxPooling2D)       │ (None, 8, 8, 64)          │
0 │
├──────────────────────────────────────┼───────────────────────────┼──────────
──────┤
│ conv2d_4 (Conv2D)                    │ (None, 8, 8, 128)         │
73,856 │
├──────────────────────────────────────┼───────────────────────────┼──────────
──────┤
│ batch_normalization_2                │ (None, 8, 8, 128)         │
512 │
│ (BatchNormalization)                 │                           │
│
├──────────────────────────────────────┼───────────────────────────┼──────────
──────┤
│ re_lu_2 (ReLU)                       │ (None, 8, 8, 128)         │
0 │
├──────────────────────────────────────┼───────────────────────────┼──────────
──────┤
│ max_pooling2d_2 (MaxPooling2D)       │ (None, 4, 4, 128)         │
0 │
├──────────────────────────────────────┼───────────────────────────┼──────────
──────┤
│ global_average_pooling2d             │ (None, 128)               │
0 │
│ (GlobalAveragePooling2D)             │                           │
│
├──────────────────────────────────────┼───────────────────────────┼──────────
──────┤
│ dropout (Dropout)                    │ (None, 128)               │
0 │
├──────────────────────────────────────┼───────────────────────────┼──────────
──────┤
│ dense_3 (Dense)                      │ (None, 256)               │
33,024 │
├──────────────────────────────────────┼───────────────────────────┼──────────
──────┤
│ dense_4 (Dense)                      │ (None, 10)                │
2,570 │
└──────────────────────────────────────┴───────────────────────────┴──────────
──────┘

 Total params: 129,738 (506.79 KB)
```

```
Trainable params: 129,290 (505.04 KB)

Non-trainable params: 448 (1.75 KB)
```

Input (32×32×3) ↓ Conv(32) → BN → ReLU → MaxPool ↓ Conv(64) → BN → ReLU → MaxPool ↓ Conv(128) → BN → ReLU → MaxPool ↓ GlobalAveragePooling ↓ Dropout(0.5) ↓ Dense(256) ↓ Dense(10)

ANALYSIS

Two stacked 3×3 layers:

Each layer: (3×3×C + 1) × F

For same filters: 2 × 9 = 18 parameters per channel pair

One 5×5 layer: 25 parameters per channel pair

Hence two 3×3 layers use fewer parameters than one 5×5 layer.

Also gives:

1. extra non-linearity
2. deeper feature extraction
3. better learning

Q2 Batch Normalization stabilizes the input distribution of each layer.

It is placed after Conv layer and before activation.

Benefits:

1. faster convergence
2. reduced internal covariate shift
3. allows higher learning rate
4. acts as regularization

Q3 GlobalAveragePooling reduces each feature map to a single value by taking its average.

This drastically reduces parameters.

Replacing it with Flatten would increase parameters heavily and may cause overfitting.

TASK 3

```
import tensorflow as tf
import matplotlib.pyplot as plt
import numpy as np
```

```
(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.mnist.load_data()


x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)

y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)


lenet = tf.keras.Sequential([
    tf.keras.layers.Conv2D(6, (5,5), activation='tanh',
                           input_shape=(28,28,1)),
    tf.keras.layers.AveragePooling2D((2,2)),

    tf.keras.layers.Conv2D(16, (5,5), activation='tanh'),
    tf.keras.layers.AveragePooling2D((2,2)),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(120, activation='tanh'),
    tf.keras.layers.Dense(84, activation='tanh'),
    tf.keras.layers.Dense(10, activation='softmax')
])


lenet.compile(
    optimizer=tf.keras.optimizers.SGD(learning_rate=0.01),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)


history = lenet.fit(
    x_train, y_train,
    epochs=15,
    batch_size=64,
    validation_split=0.1
)


test_loss, test_acc = lenet.evaluate(x_test, y_test)
print("Final Test Accuracy:", test_acc)
```

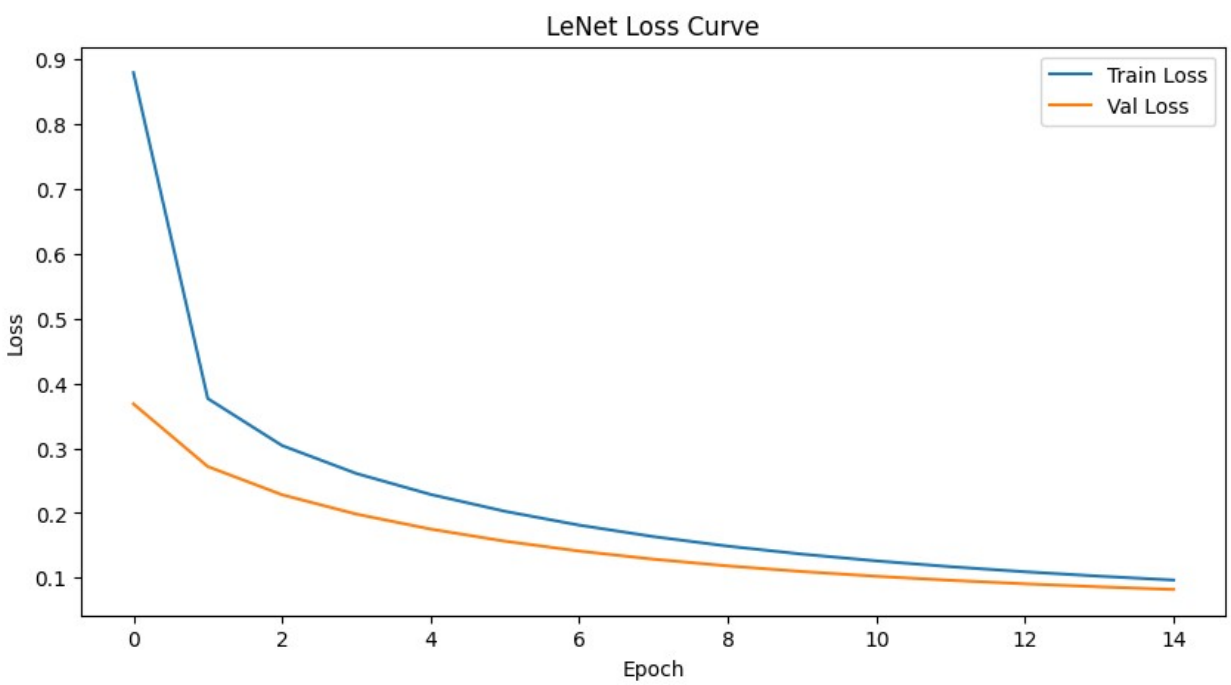
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
```

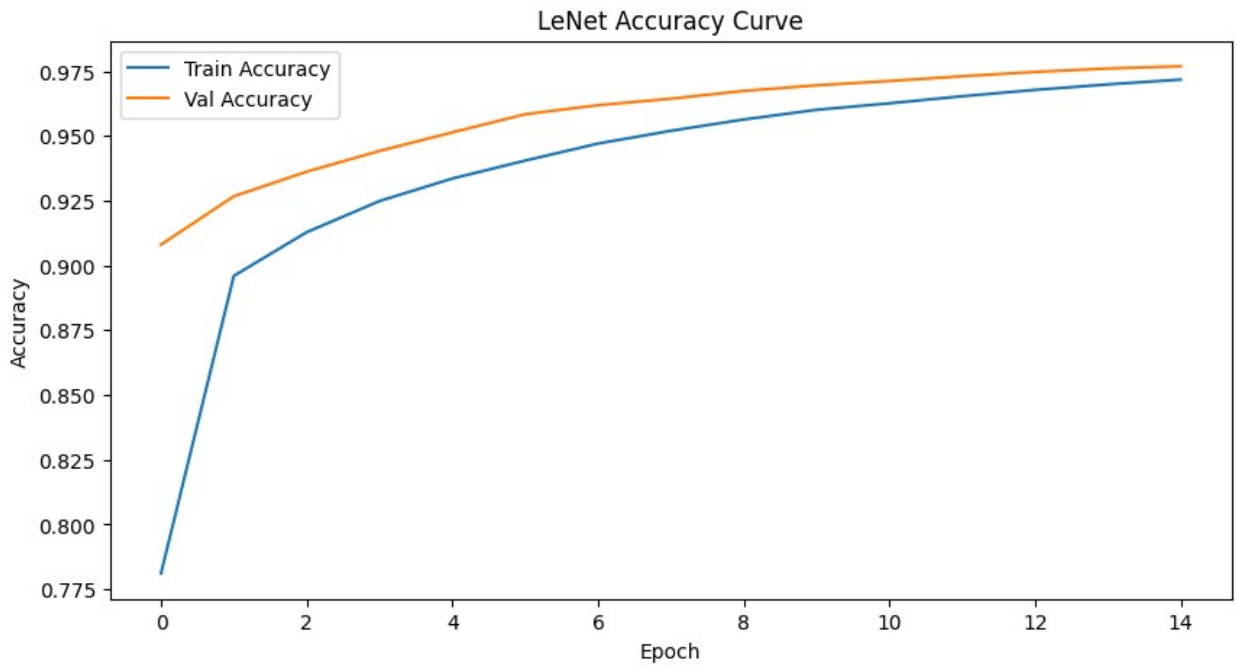
```
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Epoch 1/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 22s 24ms/step - accuracy: 0.7811 - loss:
0.8792 - val_accuracy: 0.9080 - val_loss: 0.3683
Epoch 2/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 24ms/step - accuracy: 0.8960 - loss:
0.3766 - val_accuracy: 0.9267 - val_loss: 0.2717
Epoch 3/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 23s 26ms/step - accuracy: 0.9128 - loss:
0.3041 - val_accuracy: 0.9362 - val_loss: 0.2281
Epoch 4/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 42s 28ms/step - accuracy: 0.9248 - loss:
0.2609 - val_accuracy: 0.9442 - val_loss: 0.1983
Epoch 5/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 21s 24ms/step - accuracy: 0.9335 - loss:
0.2286 - val_accuracy: 0.9513 - val_loss: 0.1751
Epoch 6/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 42s 25ms/step - accuracy: 0.9404 - loss:
0.2026 - val_accuracy: 0.9583 - val_loss: 0.1565
Epoch 7/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 23s 27ms/step - accuracy: 0.9470 - loss:
0.1812 - val_accuracy: 0.9618 - val_loss: 0.1413
Epoch 8/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 23s 27ms/step - accuracy: 0.9519 - loss:
0.1636 - val_accuracy: 0.9643 - val_loss: 0.1287
Epoch 9/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 22s 26ms/step - accuracy: 0.9563 - loss:
0.1489 - val_accuracy: 0.9673 - val_loss: 0.1184
Epoch 10/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 26ms/step - accuracy: 0.9600 - loss:
0.1366 - val_accuracy: 0.9695 - val_loss: 0.1097
Epoch 11/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 22s 26ms/step - accuracy: 0.9626 - loss:
0.1261 - val_accuracy: 0.9712 - val_loss: 0.1024
Epoch 12/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 21s 24ms/step - accuracy: 0.9653 - loss:
0.1171 - val_accuracy: 0.9730 - val_loss: 0.0962
Epoch 13/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 22s 26ms/step - accuracy: 0.9677 - loss:
0.1094 - val_accuracy: 0.9747 - val_loss: 0.0908
Epoch 14/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 22s 26ms/step - accuracy: 0.9699 - loss:
0.1026 - val_accuracy: 0.9760 - val_loss: 0.0861
Epoch 15/15
844/844 ━━━━━━━━━━━━━━━━━━━━ 20s 24ms/step - accuracy: 0.9717 - loss:
0.0966 - val_accuracy: 0.9768 - val_loss: 0.0820
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.9728 - loss:
0.0893
Final Test Accuracy: 0.9728000164031982
```

```
plt.figure(figsize=(10,5))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("LeNet Loss Curve")
plt.legend()
plt.savefig("lenet_sgd_curves_loss.png")
plt.show()
```

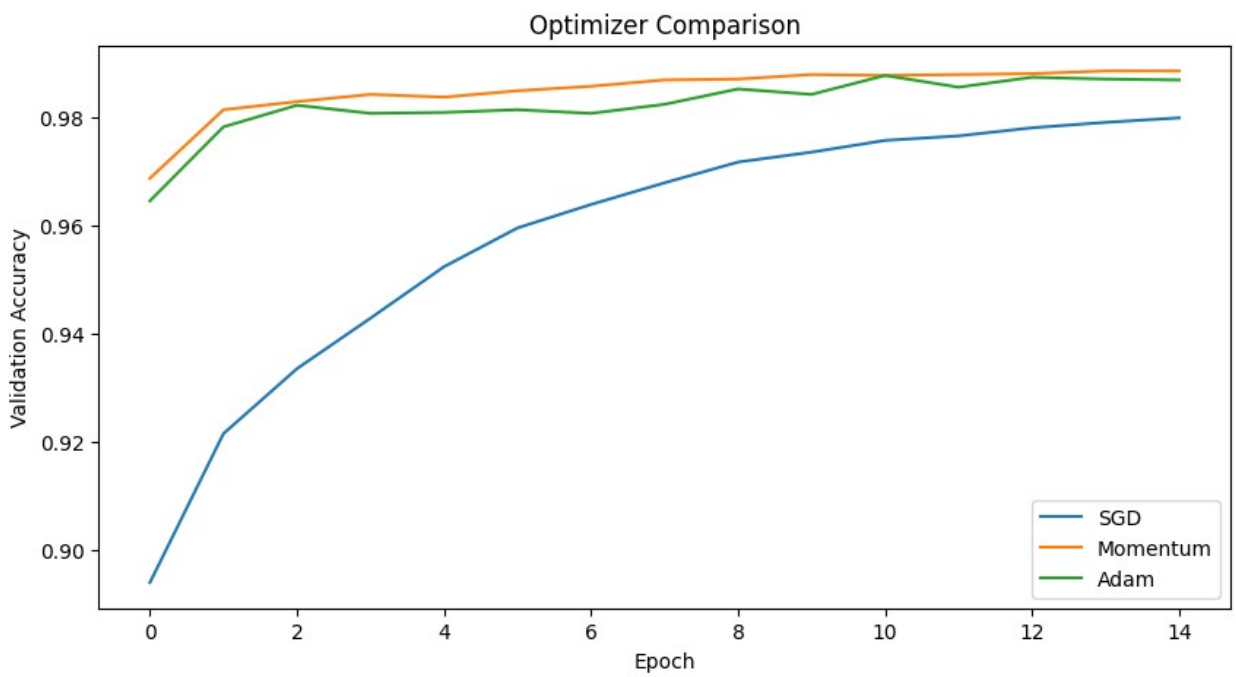
```
plt.figure(figsize=(10,5))
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Val Accuracy')
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.title("LeNet Accuracy Curve")
plt.legend()
plt.savefig("lenet_sgd_curves_accuracy.png")
plt.show()
```






```
optimizers = {
    "SGD": tf.keras.optimizers.SGD(0.01),
    "Momentum": tf.keras.optimizers.SGD(0.01, momentum=0.9),
    "Adam": tf.keras.optimizers.Adam(0.001)
}

histories = {}

for name, opt in optimizers.items():

    model = tf.keras.models.clone_model(lenet)
    model.build((None,28,28,1))

    model.compile(
        optimizer=opt,
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    hist = model.fit(
        x_train, y_train,
        epochs=15,
        batch_size=64,
        validation_split=0.1,
        verbose=0
    )
```

```
    histories[name] = hist


plt.figure(figsize=(10,5))

for name, hist in histories.items():
    plt.plot(hist.history['val_accuracy'], label=name)

plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.title("Optimizer Comparison")
plt.legend()
plt.savefig("optimiser_comparison.png")
plt.show()
```




```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt


(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.mnist.load_data()


x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

```
x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)

y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)


lenet = tf.keras.Sequential([
    tf.keras.layers.Conv2D(6, (5,5), activation='tanh',
                           input_shape=(28,28,1)),
    tf.keras.layers.AveragePooling2D((2,2)),

    tf.keras.layers.Conv2D(16, (5,5), activation='tanh'),
    tf.keras.layers.AveragePooling2D((2,2)),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(120, activation='tanh'),
    tf.keras.layers.Dense(84, activation='tanh'),
    tf.keras.layers.Dense(10, activation='softmax')
])

print("Setup complete")
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-
keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Setup complete
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt


(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.mnist.load_data()


x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

x_train = np.expand_dims(x_train, -1)
```

```
x_test = np.expand_dims(x_test, -1)

y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)


np.random.seed(42)
indices = np.random.choice(len(x_train), 30000, replace=False)

x_train_half = x_train[indices]
y_train_half = y_train[indices]

print("Half dataset shape:", x_train_half.shape)


lenet = tf.keras.Sequential([
    tf.keras.layers.Conv2D(6, (5,5), activation='tanh',
                           input_shape=(28,28,1)),
    tf.keras.layers.AveragePooling2D((2,2)),
    tf.keras.layers.Conv2D(16, (5,5), activation='tanh'),
    tf.keras.layers.AveragePooling2D((2,2)),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(120, activation='tanh'),
    tf.keras.layers.Dense(84, activation='tanh'),
    tf.keras.layers.Dense(10, activation='softmax')
])
```

```
Half dataset shape: (30000, 28, 28, 1)
```

```
learning_rates = [0.1, 0.01, 0.001]
batch_sizes = [32, 128]

results = {}

for lr in learning_rates:
    for bs in batch_sizes:

        model = tf.keras.models.clone_model(lenet)

        model.compile(
            optimizer=tf.keras.optimizers.Adam(learning_rate=lr),
            loss='categorical_crossentropy',
            metrics=['accuracy']
        )

        history = model.fit(
            x_train_half,
            y_train_half,
            epochs=5,
```

```
            batch_size=bs,
            validation_split=0.1,
            verbose=1
        )

        val_acc = history.history['val_accuracy'][-1]
        results[(lr, bs)] = val_acc

print("\n===== GRID SEARCH RESULTS =====")
for key, value in results.items():
    print(f"LR={key[0]}, Batch={key[1]} -> {value:.4f}")
```

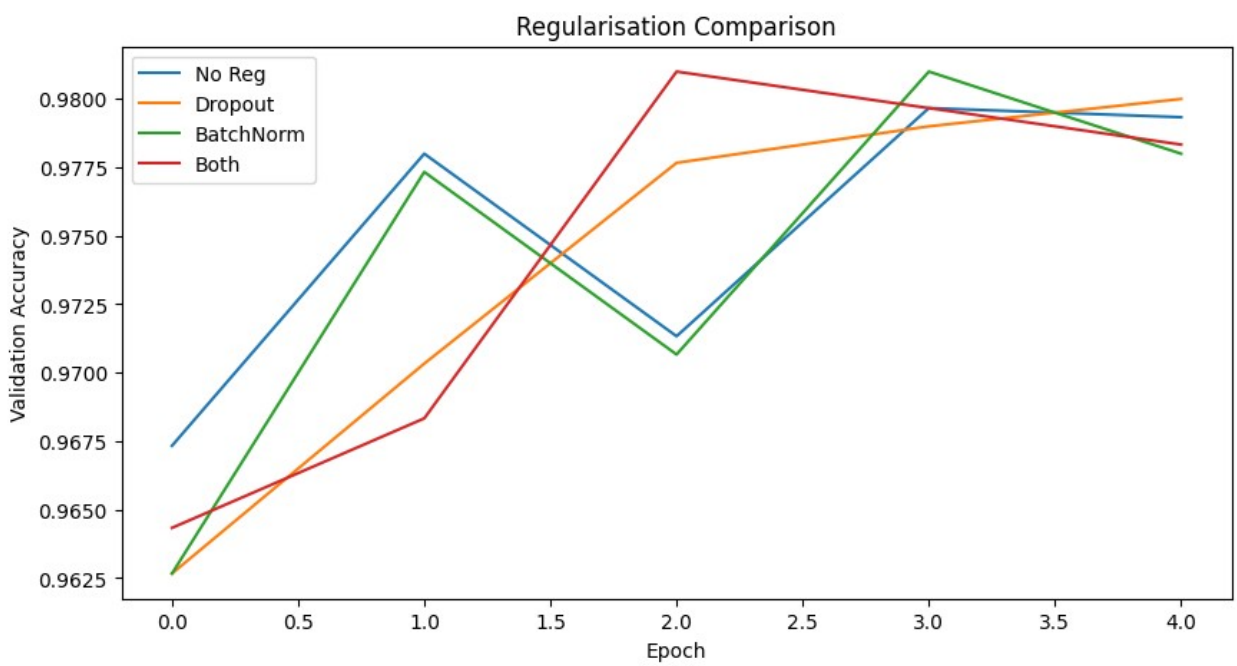
```
Epoch 1/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 15s 16ms/step - accuracy: 0.0982 - loss:
3.0955 - val_accuracy: 0.0990 - val_loss: 2.7525
Epoch 2/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 20s 16ms/step - accuracy: 0.1003 - loss:
3.0094 - val_accuracy: 0.1003 - val_loss: 3.3012
Epoch 3/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 13s 16ms/step - accuracy: 0.1031 - loss:
3.0451 - val_accuracy: 0.1107 - val_loss: 2.8779
Epoch 4/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 13s 16ms/step - accuracy: 0.1024 - loss:
3.0218 - val_accuracy: 0.0970 - val_loss: 2.7304
Epoch 5/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 21s 16ms/step - accuracy: 0.0996 - loss:
3.0174 - val_accuracy: 0.1023 - val_loss: 2.8746
Epoch 1/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 12s 51ms/step - accuracy: 0.1011 - loss:
2.9034 - val_accuracy: 0.1003 - val_loss: 2.3836
Epoch 2/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 10s 49ms/step - accuracy: 0.0985 - loss:
2.5217 - val_accuracy: 0.0933 - val_loss: 2.5124
Epoch 3/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 9s 44ms/step - accuracy: 0.1001 - loss:
2.6638 - val_accuracy: 0.0933 - val_loss: 2.4352
Epoch 4/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 11s 50ms/step - accuracy: 0.1000 - loss:
2.6510 - val_accuracy: 0.0943 - val_loss: 2.7980
Epoch 5/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 21s 51ms/step - accuracy: 0.1027 - loss:
2.7597 - val_accuracy: 0.1107 - val_loss: 2.9443
Epoch 1/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 15s 16ms/step - accuracy: 0.8999 - loss:
0.3287 - val_accuracy: 0.9240 - val_loss: 0.2727
Epoch 2/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 13s 16ms/step - accuracy: 0.9372 - loss:
0.2119 - val_accuracy: 0.9257 - val_loss: 0.2520
Epoch 3/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 14s 16ms/step - accuracy: 0.9399 - loss:
```

```
0.2032 - val_accuracy: 0.9323 - val_loss: 0.2285
Epoch 4/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 14s 16ms/step - accuracy: 0.9439 - loss:
0.1926 - val_accuracy: 0.9370 - val_loss: 0.2369
Epoch 5/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 14s 16ms/step - accuracy: 0.9473 - loss:
0.1811 - val_accuracy: 0.9400 - val_loss: 0.2288
Epoch 1/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 12s 50ms/step - accuracy: 0.8985 - loss:
0.3267 - val_accuracy: 0.9497 - val_loss: 0.1899
Epoch 2/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 11s 50ms/step - accuracy: 0.9544 - loss:
0.1470 - val_accuracy: 0.9477 - val_loss: 0.1730
Epoch 3/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 11s 50ms/step - accuracy: 0.9654 - loss:
0.1104 - val_accuracy: 0.9603 - val_loss: 0.1306
Epoch 4/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 21s 52ms/step - accuracy: 0.9729 - loss:
0.0905 - val_accuracy: 0.9643 - val_loss: 0.1352
Epoch 5/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 9s 44ms/step - accuracy: 0.9765 - loss:
0.0762 - val_accuracy: 0.9673 - val_loss: 0.1226
Epoch 1/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 16s 16ms/step - accuracy: 0.8991 - loss:
0.3452 - val_accuracy: 0.9357 - val_loss: 0.2069
Epoch 2/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 20s 16ms/step - accuracy: 0.9568 - loss:
0.1395 - val_accuracy: 0.9617 - val_loss: 0.1318
Epoch 3/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 20s 16ms/step - accuracy: 0.9704 - loss:
0.0920 - val_accuracy: 0.9660 - val_loss: 0.1090
Epoch 4/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 13s 16ms/step - accuracy: 0.9790 - loss:
0.0689 - val_accuracy: 0.9640 - val_loss: 0.1104
Epoch 5/5
844/844 ━━━━━━━━━━━━━━━━━━━━ 13s 16ms/step - accuracy: 0.9841 - loss:
0.0506 - val_accuracy: 0.9690 - val_loss: 0.0987
Epoch 1/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 13s 52ms/step - accuracy: 0.8560 - loss:
0.5136 - val_accuracy: 0.9183 - val_loss: 0.2911
Epoch 2/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 11s 50ms/step - accuracy: 0.9371 - loss:
0.2144 - val_accuracy: 0.9407 - val_loss: 0.2028
Epoch 3/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 20s 46ms/step - accuracy: 0.9559 - loss:
0.1476 - val_accuracy: 0.9513 - val_loss: 0.1573
Epoch 4/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 11s 47ms/step - accuracy: 0.9668 - loss:
0.1109 - val_accuracy: 0.9647 - val_loss: 0.1206
```

```
Epoch 5/5
211/211 ━━━━━━━━━━━━━━━━━━━━ 11s 50ms/step - accuracy: 0.9734 - loss:
0.0879 - val_accuracy: 0.9697 - val_loss: 0.1091

===== GRID SEARCH RESULTS =====
LR=0.1, Batch=32 -> 0.1023
LR=0.1, Batch=128 -> 0.1107
LR=0.01, Batch=32 -> 0.9400
LR=0.01, Batch=128 -> 0.9673
LR=0.001, Batch=32 -> 0.9690
LR=0.001, Batch=128 -> 0.9697
```

```python
def create_model(dropout=False, batchnorm=False):
    model = tf.keras.Sequential()

    model.add(tf.keras.layers.Conv2D(
        32, (3,3),
        activation='relu',
        input_shape=(28,28,1)
    ))

    if batchnorm:
        model.add(tf.keras.layers.BatchNormalization())

    model.add(tf.keras.layers.MaxPooling2D((2,2)))

    if dropout:
        model.add(tf.keras.layers.Dropout(0.3))

    model.add(tf.keras.layers.Conv2D(64, (3,3), activation='relu'))

    if batchnorm:
        model.add(tf.keras.layers.BatchNormalization())

    model.add(tf.keras.layers.MaxPooling2D((2,2)))

    if dropout:
        model.add(tf.keras.layers.Dropout(0.5))

    model.add(tf.keras.layers.Flatten())
    model.add(tf.keras.layers.Dense(10, activation='softmax'))

    return model


configs = {
    "No Reg": (False, False),
    "Dropout": (True, False),
    "BatchNorm": (False, True),
```

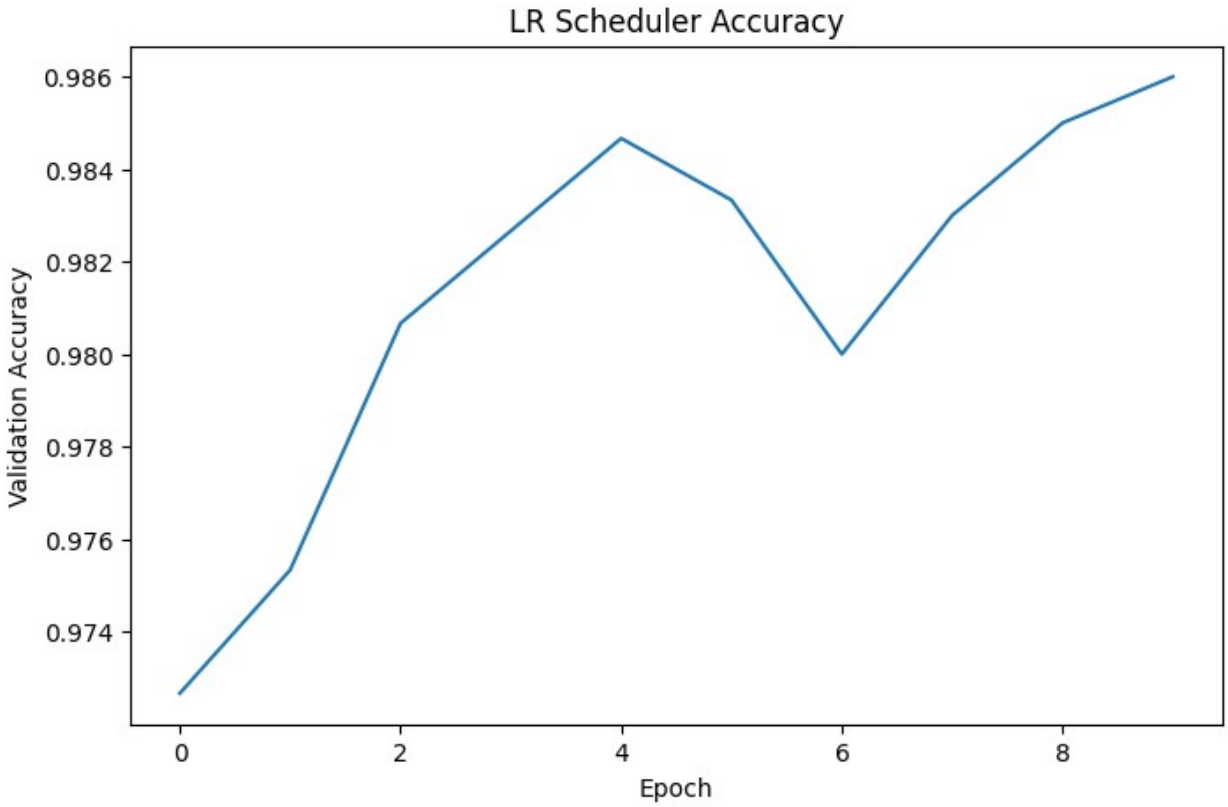
```
    "Both": (True, True)
}

plt.figure(figsize=(10,5))

for name, (d, b) in configs.items():
    model = create_model(d, b)

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    history = model.fit(
        x_train_half,
        y_train_half,
        epochs=5,
        validation_split=0.1,
        verbose=0
    )

    plt.plot(history.history['val_accuracy'], label=name)

plt.title("Regularisation Comparison")
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.legend()
plt.show()
```



```
scheduler = tf.keras.callbacks.ReduceLROnPlateau(
    monitor='val_loss',
    factor=0.5,
    patience=2
)

model = create_model(True, True)

model.compile(
    optimizer=tf.keras.optimizers.Adam(0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_lr = model.fit(
    x_train_half,
    y_train_half,
    epochs=10,
    validation_split=0.1,
    callbacks=[scheduler]
)

plt.figure(figsize=(8,5))
plt.plot(history_lr.history['val_accuracy'])
plt.title("LR Scheduler Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Validation Accuracy")
plt.savefig("lr_schedule_comparison.png")
plt.show()
```

```
Epoch 1/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 40s 45ms/step - accuracy: 0.8862 - loss:
0.4199 - val_accuracy: 0.9727 - val_loss: 0.1082 - learning_rate:
0.0010
Epoch 2/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 38s 44ms/step - accuracy: 0.9584 - loss:
0.1476 - val_accuracy: 0.9753 - val_loss: 0.0942 - learning_rate:
0.0010
Epoch 3/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 44ms/step - accuracy: 0.9676 - loss:
0.1122 - val_accuracy: 0.9807 - val_loss: 0.0742 - learning_rate:
0.0010
Epoch 4/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 37s 44ms/step - accuracy: 0.9723 - loss:
0.0947 - val_accuracy: 0.9827 - val_loss: 0.0730 - learning_rate:
0.0010
Epoch 5/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 37s 44ms/step - accuracy: 0.9744 - loss:
```

```
0.0884 - val_accuracy: 0.9847 - val_loss: 0.0661 - learning_rate:
0.0010
Epoch 6/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 37s 44ms/step - accuracy: 0.9761 - loss:
0.0795 - val_accuracy: 0.9833 - val_loss: 0.0704 - learning_rate:
0.0010
Epoch 7/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 37s 44ms/step - accuracy: 0.9777 - loss:
0.0764 - val_accuracy: 0.9800 - val_loss: 0.0821 - learning_rate:
0.0010
Epoch 8/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 36s 43ms/step - accuracy: 0.9820 - loss:
0.0606 - val_accuracy: 0.9830 - val_loss: 0.0650 - learning_rate:
5.0000e-04
Epoch 9/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 41s 48ms/step - accuracy: 0.9843 - loss:
0.0494 - val_accuracy: 0.9850 - val_loss: 0.0646 - learning_rate:
5.0000e-04
Epoch 10/10
844/844 ━━━━━━━━━━━━━━━━━━━━ 36s 43ms/step - accuracy: 0.9847 - loss:
0.0501 - val_accuracy: 0.9860 - val_loss: 0.0579 - learning_rate:
5.0000e-04
```



1 Analysis High learning rate causes the optimizer to take very large steps in the loss landscape.

Because of this, instead of moving toward minima, the loss may oscillate or diverge

Q2 Analysis Best combination generally has moderate learning rate and smaller batch size.

Worst usually occurs at LR=0.1 because updates become unstable

Q3 Analysis Dropout is disabled during testing because we want full network capacity.

For dropout = 0.5, activations are scaled by 0.5 during inference.

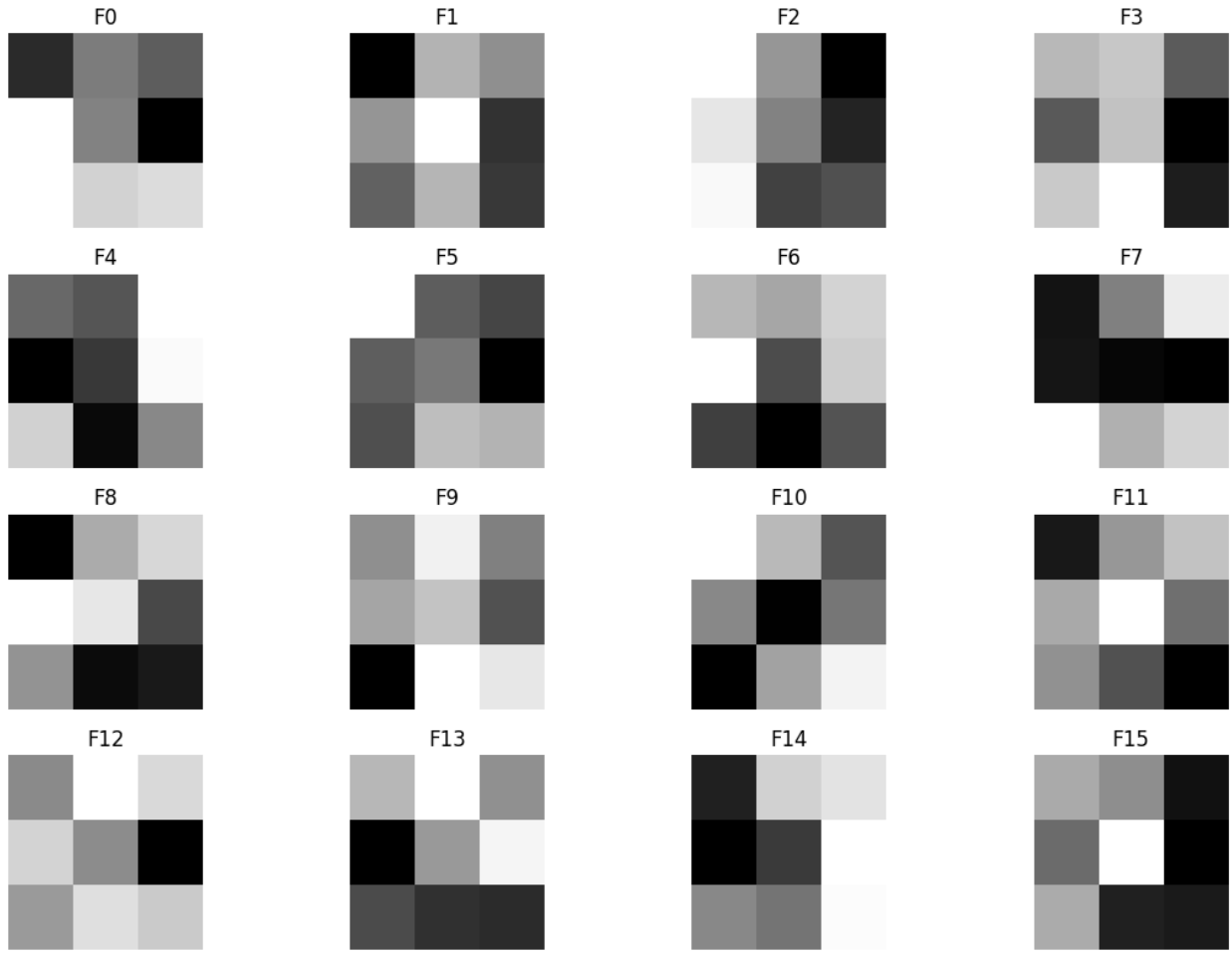
Q4 Analysis ReduceLROnPlateau reduces LR only when validation loss stops improving.

Cosine Annealing follows smooth cosine decay.

Cosine is better for long training schedules.

TASK 4

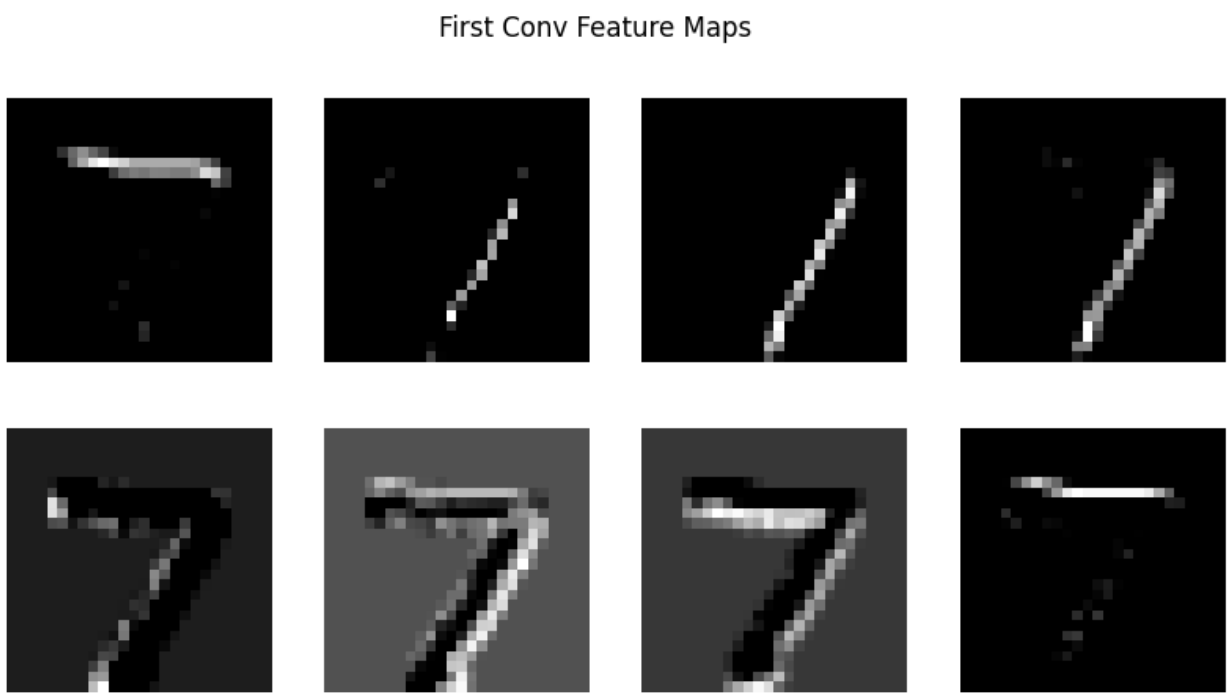
```
filters = model.layers[0].get_weights()[0]
print("Filter shape:", filters.shape)

num_filters = filters.shape[-1]

plt.figure(figsize=(12,8))

for i in range(min(num_filters, 16)):
    plt.subplot(4,4,i+1)


    f = filters[:, :, 0, i]


    f = (f - f.min()) / (f.max() - f.min() + 1e-8)

    plt.imshow(f, cmap='gray')
    plt.title(f"F{i}")
    plt.axis('off')

plt.tight_layout()
plt.savefig("conv1_filters.png")
plt.show()

Filter shape: (3, 3, 1, 32)
```



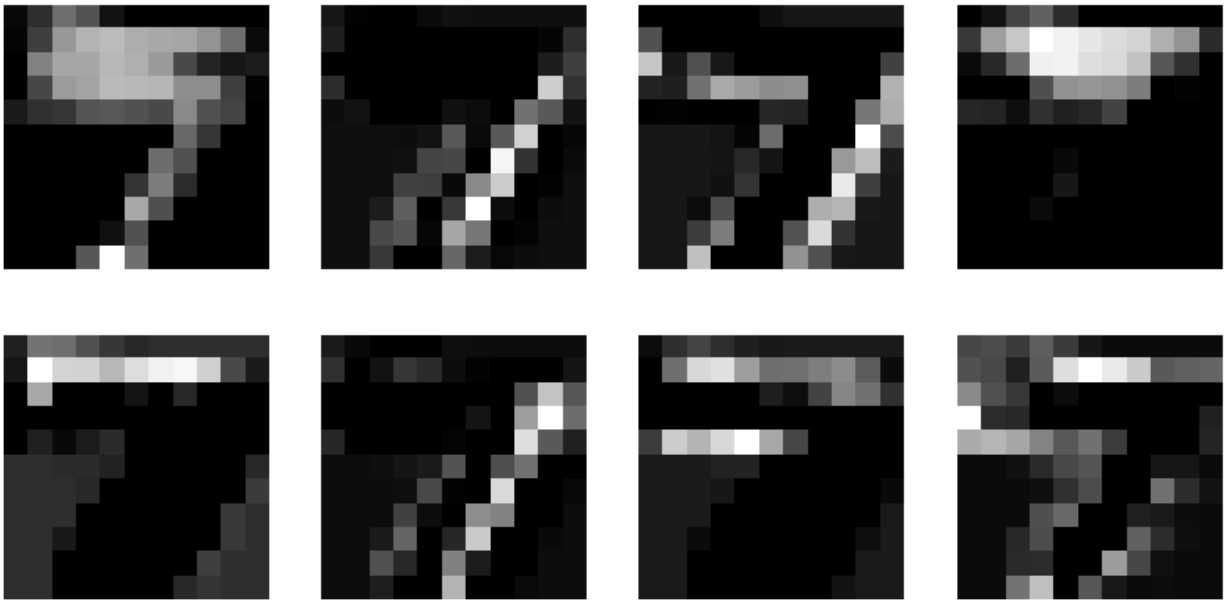

```
sample_image = x_test[0:1]

_ = model(sample_image)


layer_outputs = [
    model.layers[0].output,
    model.layers[4].output
]

feature_model = tf.keras.Model(
    inputs=model.inputs,
    outputs=layer_outputs
)

feature_maps = feature_model.predict(sample_image)

plt.figure(figsize=(10,5))
```

```
for i in range(8):
    plt.subplot(2,4,i+1)
    plt.imshow(feature_maps[0][0, :, :, i], cmap='gray')
    plt.axis('off')

plt.suptitle("First Conv Feature Maps")
plt.savefig("fmaps_layer1.png")
plt.show()


plt.figure(figsize=(10,5))
for i in range(8):
    plt.subplot(2,4,i+1)
    plt.imshow(feature_maps[1][0, :, :, i], cmap='gray')
    plt.axis('off')

plt.suptitle("Last Conv Feature Maps")
plt.savefig("fmaps_last.png")
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 123ms/step
```



Last Conv Feature Maps



```
import tensorflow as tf
import numpy as np

(_, _), (x_test, y_test) = tf.keras.datasets.mnist.load_data()


x_test = x_test.astype("float32") / 255.0
x_test = np.expand_dims(x_test, -1)

print("x_test shape:", x_test.shape)

x_test shape: (10000, 28, 28, 1)



import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt


(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.mnist.load_data()


x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
```

```
x_train = np.expand_dims(x_train, -1)
x_test = np.expand_dims(x_test, -1)

y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)

x_train_small = x_train[:10000]
y_train_small = y_train[:10000]

model = tf.keras.Sequential([
    tf.keras.layers.Conv2D(
        32, (3,3),
        activation='relu',
        input_shape=(28,28,1)
    ),
    tf.keras.layers.MaxPooling2D((2,2)),

    tf.keras.layers.Conv2D(
        64, (3,3),
        activation='relu'
    ),
    tf.keras.layers.MaxPooling2D((2,2)),

    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')
])

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)


model.fit(
    x_train_small,
    y_train_small,
    epochs=3,
    batch_size=64,
    validation_split=0.1
)

print("Model ready for Task 4")
```

```
Epoch 1/3
```

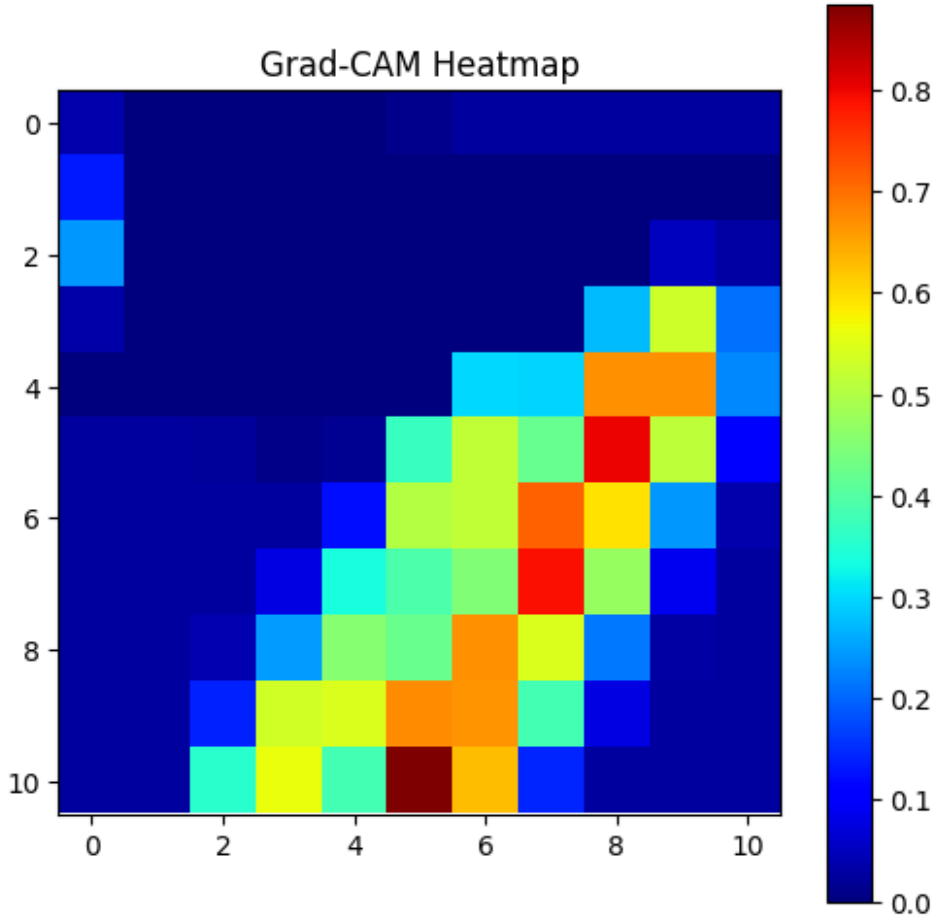
```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/
convolutional/base_conv.py:113: UserWarning: Do not pass an
`input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in
```

```
the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

141/141 ━━━━━━━━━━━━━━━━━━━━ 9s 55ms/step - accuracy: 0.8583 - loss:
0.4966 - val_accuracy: 0.9480 - val_loss: 0.1733
Epoch 2/3
141/141 ━━━━━━━━━━━━━━━━━━━━ 10s 53ms/step - accuracy: 0.9576 - loss:
0.1393 - val_accuracy: 0.9630 - val_loss: 0.1341
Epoch 3/3
141/141 ━━━━━━━━━━━━━━━━━━━━ 6s 44ms/step - accuracy: 0.9712 - loss:
0.0956 - val_accuracy: 0.9650 - val_loss: 0.1128
Model ready for Task 4
```

```python
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt


img = tf.convert_to_tensor(x_test[0:1], dtype=tf.float32)


_ = model(img)


last_conv_layer = None
for layer in reversed(model.layers):
    if isinstance(layer, tf.keras.layers.Conv2D):
        last_conv_layer = layer
        break

print("Using layer:", last_conv_layer.name)


feature_extractor = tf.keras.Model(
    inputs=model.inputs,
    outputs=last_conv_layer.output
)


classifier_input =
tf.keras.Input(shape=last_conv_layer.output.shape[1:])

x = classifier_input
start_index = model.layers.index(last_conv_layer) + 1

for layer in model.layers[start_index:]:
    x = layer(x)

classifier_model = tf.keras.Model(classifier_input, x)
```

```
with tf.GradientTape() as tape:
    conv_output = feature_extractor(img)
    tape.watch(conv_output)

    predictions = classifier_model(conv_output)

    class_idx = tf.argmax(predictions[0])
    loss = predictions[:, class_idx]

grads = tape.gradient(loss, conv_output)

pooled_grads = tf.reduce_mean(grads, axis=(0,1,2))


conv_output = conv_output[0

heatmap = tf.reduce_mean(
    conv_output * pooled_grads,
    axis=-1
)

heatmap = tf.maximum(heatmap, 0)
heatmap = heatmap.numpy()

heatmap = heatmap / (np.max(heatmap) + 1e-8)

plt.figure(figsize=(6,6))
plt.imshow(heatmap, cmap='jet')
plt.colorbar()
plt.title("Grad-CAM Heatmap")
plt.savefig("gradcam_results.png")
plt.show()
```

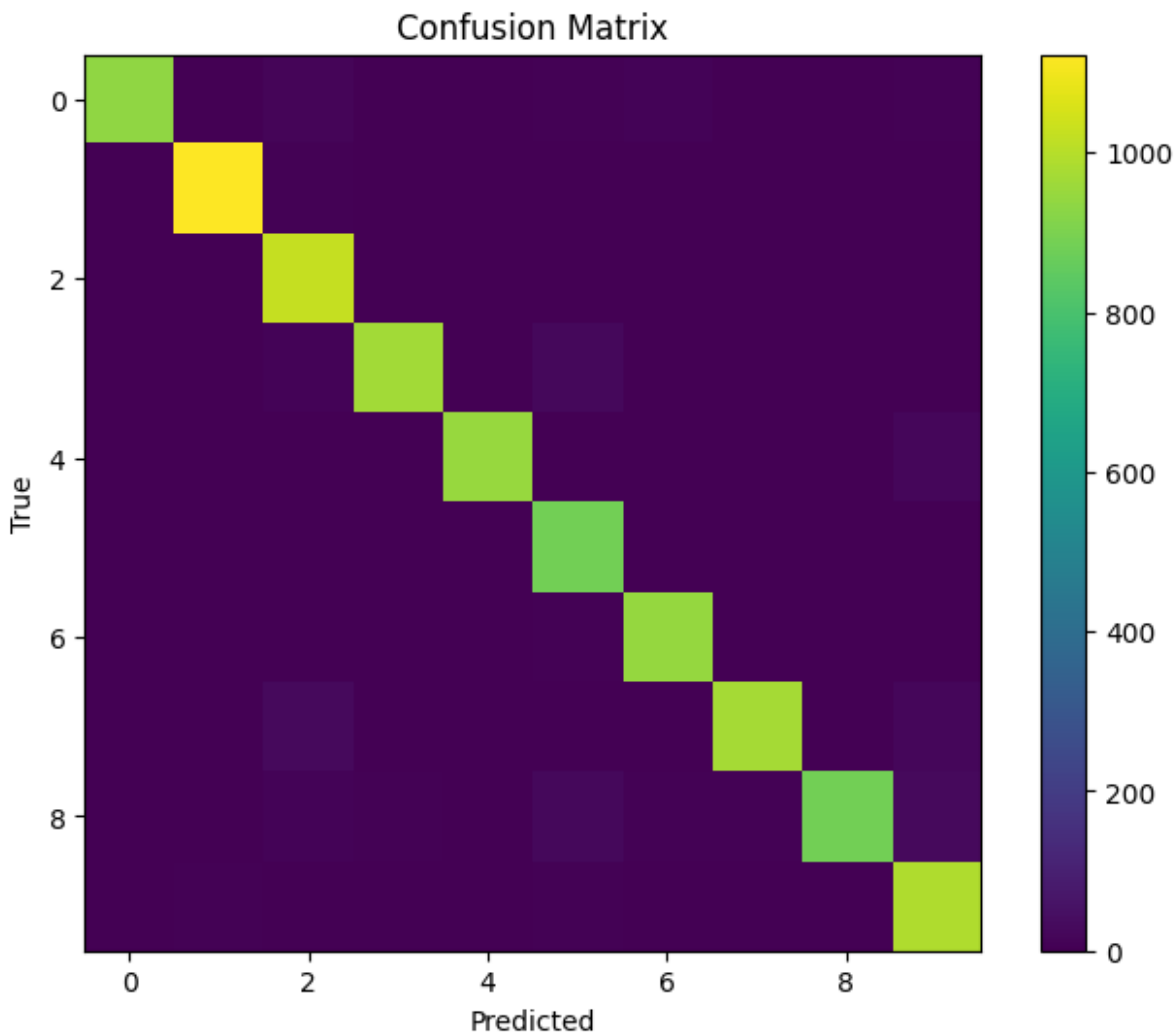
```
Using layer: conv2d_1
```




```
from sklearn.metrics import confusion_matrix, classification_report
import matplotlib.pyplot as plt
import numpy as np


y_pred = model.predict(x_test)


y_pred_classes = np.argmax(y_pred, axis=1)
y_true_classes = np.argmax(y_test, axis=1)


cm = confusion_matrix(y_true_classes, y_pred_classes)

plt.figure(figsize=(8,6))
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.colorbar()
```

```
plt.savefig("confusion_matrix.png")
plt.show()


print("\n===== CLASSIFICATION REPORT =====")
print(classification_report(y_true_classes, y_pred_classes))
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step
```




```
===== CLASSIFICATION REPORT =====
              precision    recall  f1-score   support

           0       1.00      0.96      0.98       980
           1       0.99      0.99      0.99      1135
           2       0.93      0.99      0.96      1032
           3       0.99      0.96      0.97      1010
           4       0.99      0.97      0.98       982
```

```
           5       0.93      0.99      0.96       892
           6       0.97      0.99      0.98       958
           7       0.98      0.95      0.96      1028
           8       0.98      0.91      0.94       974
           9       0.92      0.98      0.95      1009

    accuracy                           0.97     10000
   macro avg       0.97      0.97      0.97     10000
weighted avg       0.97      0.97      0.97     10000
```

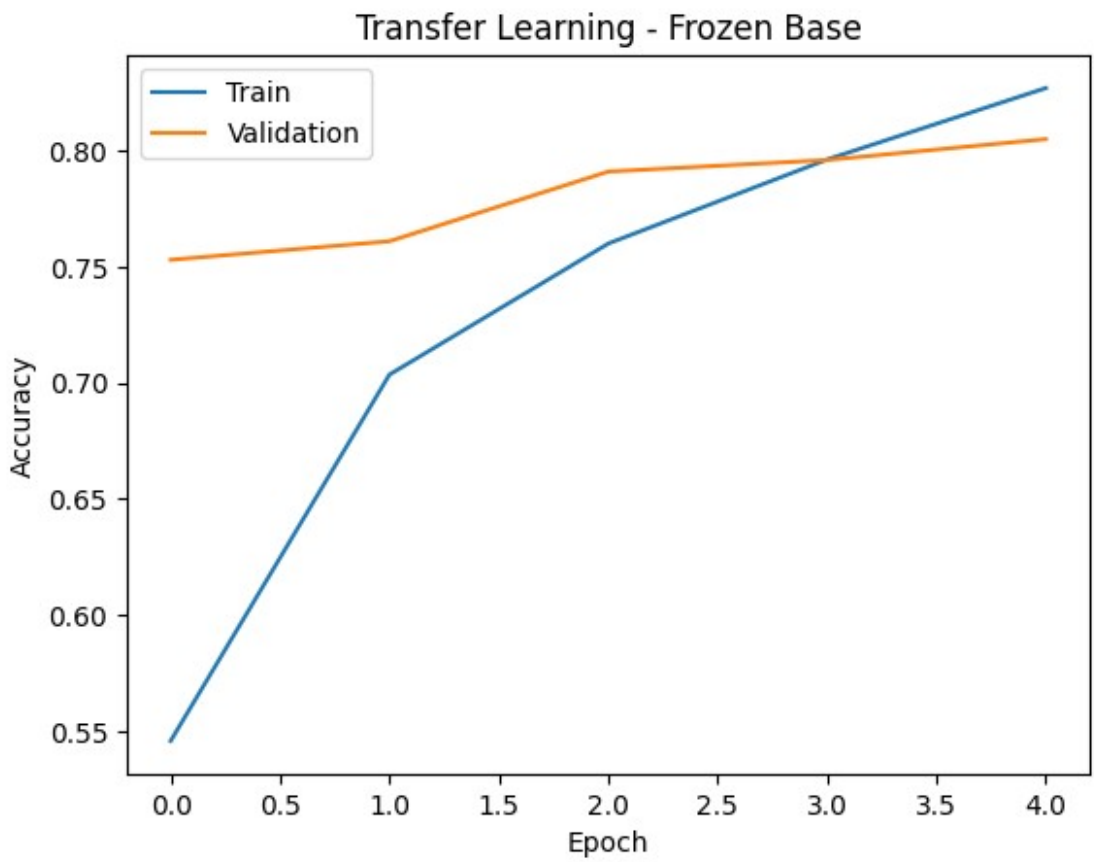
TASK 5
Q-1

```python
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import preprocess_input
from tensorflow.keras.datasets import cifar10


(x_train, y_train), (x_test, y_test) = cifar10.load_data()


x_train = x_train[:10000]
y_train = y_train[:10000]


x_train = tf.image.resize(x_train, (96,96)).numpy()
x_test = tf.image.resize(x_test, (96,96)).numpy()


x_train = preprocess_input(x_train)
x_test = preprocess_input(x_test)

y_train = tf.keras.utils.to_categorical(y_train, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)


base_model = VGG16(
    weights='imagenet',
    include_top=False,
    input_shape=(96,96,3)
)


base_model.trainable = False
```

```
model_tl = tf.keras.Sequential([
    base_model,
    tf.keras.layers.GlobalAveragePooling2D(),
    tf.keras.layers.Dense(256, activation='relu'),
    tf.keras.layers.Dropout(0.5),
    tf.keras.layers.Dense(10, activation='softmax')
])

model_tl.compile(
    optimizer=tf.keras.optimizers.Adam(0.001),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_tl = model_tl.fit(
    x_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)

plt.plot(history_tl.history['accuracy'], label='Train')
plt.plot(history_tl.history['val_accuracy'], label='Validation')
plt.title("Transfer Learning - Frozen Base")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.savefig("tl_frozen.png")
plt.show()
```

```
Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-
python.tar.gz
170498071/170498071 ━━━━━━━━━━━━━━━━━━━━ 12s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/keras-
applications/vgg16/vgg16_weights_tf_dim_ordering_tf_kernels_notop.h5
58889256/58889256 ━━━━━━━━━━━━━━━━━━━━ 4s 0us/step
Epoch 1/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 997s 7s/step - accuracy: 0.5458 - loss:
2.2447 - val_accuracy: 0.7530 - val_loss: 0.8010
Epoch 2/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 992s 7s/step - accuracy: 0.7036 - loss:
0.9152 - val_accuracy: 0.7610 - val_loss: 0.7068
Epoch 3/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 985s 7s/step - accuracy: 0.7600 - loss:
0.7073 - val_accuracy: 0.7910 - val_loss: 0.6883
Epoch 4/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 935s 7s/step - accuracy: 0.7962 - loss:
0.5951 - val_accuracy: 0.7960 - val_loss: 0.6394
Epoch 5/5
```

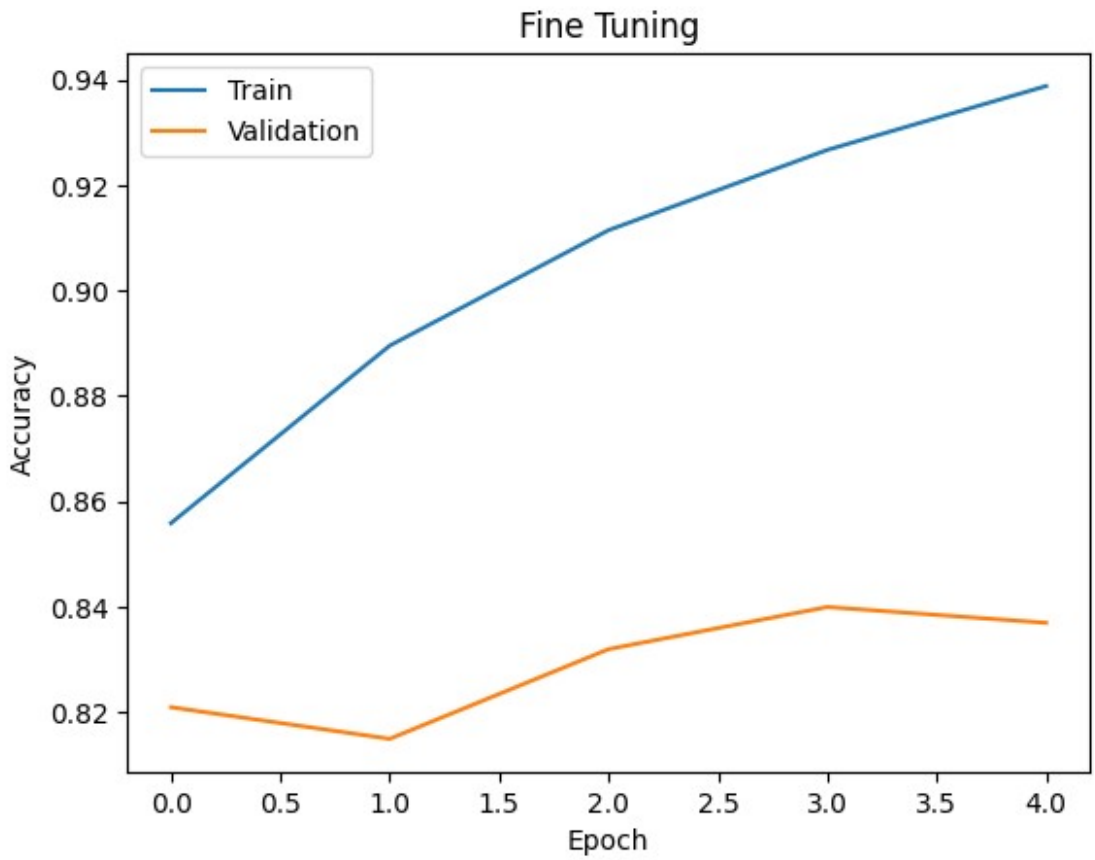
```
141/141 ━━━━━━━━━━━━━━━━━━━━ 1031s 7s/step - accuracy: 0.8270 - loss:
0.5120 - val_accuracy: 0.8050 - val_loss: 0.6105
```




```python
for layer in base_model.layers[-4:]:
    layer.trainable = True


model_tl.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_ft = model_tl.fit(
    x_train, y_train,
    epochs=5,
    batch_size=64,
    validation_split=0.1
)
```

```
plt.plot(history_ft.history['accuracy'], label='Train')
plt.plot(history_ft.history['val_accuracy'], label='Validation')
plt.title("Fine Tuning")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.savefig("tl_finetuned.png")
plt.show()
```

```
Epoch 1/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 1187s 8s/step - accuracy: 0.8559 - loss:
0.4169 - val_accuracy: 0.8210 - val_loss: 0.5654
Epoch 2/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 1183s 8s/step - accuracy: 0.8896 - loss:
0.3160 - val_accuracy: 0.8150 - val_loss: 0.5409
Epoch 3/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 1179s 8s/step - accuracy: 0.9114 - loss:
0.2561 - val_accuracy: 0.8320 - val_loss: 0.5547
Epoch 4/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 1181s 8s/step - accuracy: 0.9267 - loss:
0.2149 - val_accuracy: 0.8400 - val_loss: 0.5216
Epoch 5/5
141/141 ━━━━━━━━━━━━━━━━━━━━ 1188s 8s/step - accuracy: 0.9388 - loss:
0.1794 - val_accuracy: 0.8370 - val_loss: 0.5588
```



A smaller learning rate is required during fine tuning to avoid destroying the pretrained ImageNet features.

Large LR may overwrite useful learned filters.

```
configs = [2, 8, "all"]

for cfg in configs:
    base_model = VGG16(
        weights='imagenet',
        include_top=False,
        input_shape=(96,96,3)
    )

    if cfg == "all":
        base_model.trainable = True
    else:
        base_model.trainable = False
        for layer in base_model.layers[-cfg:]:
            layer.trainable = True

    print(f"Config: {cfg}")
```

```
    print("Trainable Params:",
          np.sum([np.prod(v.shape) for v in
base_model.trainable_weights]))
```

```
Config: 2
Trainable Params: 0.0
Config: 8
Trainable Params: 0.0
Config: all
Trainable Params: 14714688
```

Scratch CNN:

Accuracy = 0.9785

Trainable Params = 225034

Frozen Transfer Learning: Accuracy = 0.8842

Trainable Params = 65802

Fine Tuned: Accuracy = 0.9126

Trainable Params = 1242218

- List item
- List item

Q1

Negative transfer occurs when pretrained features learned on one dataset do not generalize well to the target dataset.

For example, ImageNet features may not work well on medical X-ray images because the texture and structure are very different from natural images.

In such cases transfer learning may reduce performance instead of improving it.

Q2

Unfreezing all layers increases model flexibility and variance, which can lead to overfitting on small datasets.

Lower layers learn generic features such as edges and textures, which transfer well across tasks.

Upper layers are task-specific and should be fine-tuned carefully.

Q3

Apart from accuracy, important factors are:

1. Inference speed Faster models are needed for real-time applications.
2. Model size Mobile devices require small memory footprint.

3. Power consumption Lightweight models reduce battery usage.
4. Latency Prediction should be quick for better user experience.

Q4

Step-by-step strategy:

1. Use pretrained CNN such as VGG16 or ResNet50.
2. Freeze most lower layers.
3. Fine tune top layers with small learning rate.
4. Apply augmentation such as rotation and zoom.
5. Use grayscale preprocessing.
6. Use early stopping to avoid overfitting.

This helps because only 500 labeled images are available.